
ARTICLE

Cognitive Resource Allocation System with Deep Learning-Based Offloading for Enhanced Mobile Computing Performance

M. Nalini^{1,*} and Sivakumar Vengusamy²

¹Assistant Professor, Department of CT and DS, Sri Krishna Arts and Science College, Coimbatore, India.

²Associate Professor, School of Computing, Asia Pacific University of Technology and Innovation (APU), Malaysia.

*Corresponding Author: M. Nalini. Email: nalini1911@gmail.com

Received: 10 June Year; Accepted: 12 October 2024; Published: 30 December 2024

ABSTRACT: Mobile devices with sophisticated hardware and software have emerged as the primary computing platforms for the day-to-day activities of individuals. However, computation on these mobile app developments is still inhibited by the limited battery power, restricted memory, and limited resources. To address this problem, the augmentation of mobile devices with resources from surrounding devices or with cloud-based resources has gained popularity in recent years. Existing solutions that follow this approach and offload computationally intensive tasks already yield great results for specific use cases. Unfortunately, most of these solutions are tailored to specific operating systems or programming languages, and do not support the flexible usage of resources. So, the optimization of the resources, the bat optimization-based Resource Optimizer, is introduced, which optimizes the resource constraints of mobile devices. For optimizing the resources of mobile devices, a model based on a centralized broker-node is proposed in this research work. At the same time, the limited battery power of a mobile device is solved by using computational offloading, by executing some components of a mobile application in the cloud. This work presents a novel offloading algorithm called Deep Neural Network (DNN) is presented. The DNN algorithm will offload as many tasks as possible to the cloud when the network transmission bandwidth is high, thereby improving the total execution time of all tasks and minimizing the energy use of the mobile device. This research work provides a feasible solution for the development of resource-intensive mobile applications and their accessibility by mobile users regardless of the resource scarcity of mobile devices.

KEYWORDS: Mobile Edge Clouding (MEC); Mobile Resource Augmentation; Bat optimization-based resource optimizer, Offloading, Deep Neural Networks; Raspberry PI controller; Arduino; MNO cloud.

1 Introduction

Nowadays, smartphones have become one of the gadgets of all people due to their technological advancements, which led to the fast development of resource-intensive applications for watching videos, web surfing, interactive gaming, augmented reality, and location tracking, etc. [1] These kinds of applications constantly require intensive computational power, memory, and battery. In the modern era, a novel approach of a computing model is made possible with the available technologies like high-end wireless networks, universal devices for communication, Geographical positioning, the internet, etc., have a great impact and form the base for Mobile Cloud Computing (MCC) [2]. It helps the users by giving web access to an infinite computational power and memory space. The cloud



technology eases the overload of complexities, such as storage and computation, that are used in real time as a utility.

Mobile devices have become the global interface for users to access services online and applications in the cloud. These are limited to primary techniques, such as either the applications are installed on the device, or the same is accessed via a server from mobile devices. The major setback for these techniques is the lack of flexibility in service interaction, which limits the chances for optimization to provide better performance. The mobile devices are often subjected to resource scarcity, mainly the power and the complex computational needs. This made the MCC come into existence, which provides resources from a farther end to enable offloading of complex computational tasks and thereby addressing the problem of scarcity in resources. In spite of these advances, the users are not able to make the fullest use of the technology because of the Hubs present as intermediate, the exponential increase in traffic of the network, and the non-availability of high-end end connectivity between the device and the server in a typical MCC model [3].

The motivation for this interest is the deficiency of computing resources and their manageability. In addition, distributed databases and multitier applications handling hundreds and thousands of requests demand more computing resources. More computing resource means more power and complexity, thus increasing the cost of the solutions. Consequently, to overcome the drawbacks of mobile cloud computing, MEC [4] emerges, which provides cloud computing and services closer to the mobile device. Organizations are seeking outsourced solutions, such as those provided by MCC and Edge Computing (EC). EC solutions include MEC, cloudlet-based computing, and fog computing (FC). Clearly, there is a difference between MCC and EC approaches. However, both serve the same purpose of providing computing resources for resource-limited devices. In MCC, computations are performed in the core network (cloud). In MEC, computations are performed on the network edge, maintaining closer proximity to the user, preferably in the base station of the radio access network (RAN) [5].

Edge computing is a middleware between mobile devices and cloud servers, thereby providing cloud resources at the edge of the network with minimum delay. The MEC is a technology which works to bring closer to the users of applications and content. (e.g., at base stations, access points, and aggregation networks) to minimize the latency and to maximize the operations in a network, ensuring the service delivery. Hence, this is a new theory for providing complex computations in the edge networks closer to the mobile devices in order to ensure speed in execution time of the applications and also to minimize the energy consumption of the devices [6].

MCC has been coined to augment the capability of such handheld devices. MCC empowers the application processing capabilities of the mobile devices by offloading computationally intensive applications to resource-rich distributed application processing systems. But the key challenge lies in the seamless distribution of the applications on the distributed application processing cloud platforms. Researchers have explored varied resources for the augmentation of mobile device capabilities. There has been a substantial rise in requests for multimedia data, such as videos and games, over mobile phones. If at all there is a significant growth in improvisation of network capacity, the device hardware has always been hindrances such as poor resources, insecurity and unstable connections, and energy consumption. The non-availability of adequate resources has always been a great constraint for many applications [7].

Resource Augmentation is one of the best augmentation models that uses high resources in clouds to improve the overall performance of the Mobile Cloud Computing. This aims for the execution of tasks that require complex computation and the applications that tend to use a large number of resources. The augmented devices are particularly focused on executing large computations and storing very large data, which are normally beyond their ability, with a minimum footprint and security. Researchers utilize varied cloud-based computing resources (e.g., distant

clouds and nearby mobile nodes) to meet various computing requirements of mobile users [8]. Therefore, the computation offloading technology in the MEC can effectively enhance the computation capability of mobile devices.

The concept of offloading the computational tasks has got a large attention among the research community in the domains of cloud and mobile computing. These aim to remove the major pitfalls of a conventional mobile computing by augmenting the capability with other resources outside. In general, the concepts of offloading and augmentation are synonymous and are used to minimize the limitations of a mobile device in terms of computational complexity, Storage, and Power consumption. Partitioning and offloading some parts of mobile applications is a promising method of saving time and energy for mobile devices with constrained resources. Most of the current research work has focused on how to design effective offloading systems to save time and energy while giving limited consideration to enabling techniques. Moreover, researchers have paid attention to different computational resources in offloading frameworks. So, the MEC reduces network operational cost and also upgrades the quality of service (QoS) by outsourcing computation tasks to the network edges [9].

In this research work, the optimization of the resources, the bat optimization-based Resource Optimizer, is introduced, which optimizes the resource constraints of mobile devices. A novel offloading algorithm called Deep Neural Network (DNN) is presented. The DNN algorithm will offload as many tasks as possible to the cloud when the network transmission bandwidth is high, thereby improving the total execution time of all tasks and minimizing the energy use of the mobile device.

This work is framed as follows: Section I introduces mobile edge computing, offloading, and resource augmentation concepts; Section II presents the related works for mobile resource augmentation. In section III, the proximate computing framework for mobile resource augmentation is discussed and explains the intelligent inventory checker application, which is the implementation of the proposed framework; In section IV discusses the results from the computing entities are discussed. Section V presents the conclusions and future work.

2 Related Works

Kovachev et al [10] proposed a technique termed MACS (Mobile Augmentation Cloud Services). This makes it possible for a mobile client to run an application in the cloud by extending the Android application. The standard procedures of Android development are used. The use of middleware is to lift the heavy task of adaptive application partitioning, the monitoring of the resources that are used, and the offloading of complex computations. These work both as a normal mobile application and also it can also make use of the computational resources available in a far place in a transparent manner. A couple of prototypes have been developed, which clearly indicate that the energy saving is up to 95% when the offloading technique is used for long and complex computational tasks, when compared with that of the available methods, where local execution alone takes place.

Kemp et al [11] presented the first practical implementation idea for Android: the cuckoo framework, which simplifies the development of smartphone applications that benefit from computation offloading and provides a dynamic runtime system that can, at runtime, decide whether a part of an application will be executed locally or remotely. They evaluated the Cuckoo framework with two real-world smartphone applications, an object recognition application and a distributed augmented reality smartphone game, and showed that little work was required to enable computation offloading for these applications using the Cuckoo framework.

Orsini et al [12] presented the evaluation of Cloud Aware, a context-adaptive mobile middleware for MCC as well as MEC that supports automated context adaptation by linking the distribution features of mobile middleware with context-aware self-adaptation techniques. In particular, present a system software infrastructure and a data mining process that facilitate the development of elastic,

scalable, and context-adaptive mobile applications and present their evaluation using real usage data provided by the Nokia Mobile Data Challenge (MDC) dataset.

Quoc et al [13] proposed an optimization framework for computation offloading and resource allocation for MEC with multiple servers. This work adopted a suboptimal approach by splitting the original problem into two parts: (i) computation offloading decision and (ii) joint resource allocation. To enable distributed computation offloading, two matching algorithms are investigated. Moreover, the transmit power of offloading users is found using a bisection method with approximate inter-cell interference, and the computation resources allocated to offloading users are achieved via the duality approach. Simulation results validate that the proposed framework can significantly improve the percentage of offloading users and reduce the system overhead with respect to the existing schemes.

Shiraz et al [14] proposed a novel distributed Energy Efficient Computational Offloading Framework (EECOF) for the processing of intensive mobile applications in Mobile Cloud Computing (MCC). The framework focuses on leveraging application processing services of cloud datacenters with minimal instances of computationally intensive component migration at runtime. As a result, the size of data transmission and energy consumption cost is reduced in computational offloading for MCC. Hence, EECOF provides an energy-efficient application layer solution for computational offloading in MCC.

Deng et al [15] proposed a novel offloading system to design robust offloading decisions for mobile services. This approach considers the dependency relations among component services and aims to optimize execution time and energy consumption of executing mobile services. To this end, we also introduce a mobility model and a trade-off fault-tolerance mechanism for the offloading system. A genetic algorithm (GA) based offloading method is then designed and implemented after carefully modifying parts of a generic GA to match these special needs for the stated problem.

Shiraz et al [16] proposed an active service migration (ASM) framework for computational offloading to cloud datacenters, which employs a lightweight procedure for the deployment of a runtime distributed platform. The proposed framework employs a coarse granularity level and simple developmental and deployment procedures for computational offloading in MCC. ASM is evaluated by benchmarking a prototype application on the Android devices in the real MCC environment. It is found that the turnaround time of the application reduces up to 45 % and ECC of the application reduces up to 33 % in ASM-based computational offloading, as compared to traditional offloading techniques, which shows the lightweight nature of the proposed framework for computational offloading.

Kumar and Malik [17] proposed a context-based computation augmentation framework to balance the competing goals. It incorporates individual contextual demands in its optimization engine to dynamically adjust its offloading decisions. In addition, it also proposed an adaptive virtual machine (VM) provisioning technique for efficient utilization of cloud resources and maintaining the QoS. Evaluate the design space of the proposed system by extending the CloudSim simulator. The experimental results show that the proposed system can provide contextual augmentation services with QoS and low costs.

Nir et al [18] focused on and proposed a centralized broker-node architecture which manages resource monitoring on behalf of all mobile devices. In the centralized architecture, mathematical models for the task scheduling problem in the local resources case and the mobile cloud computing case were proposed to optimally minimize the total energy consumption across all mobile devices. A generalized model for the task scheduling problem was proposed. The models found optimal solutions for the centralized task scheduling problems, and an improvement in the total costs was observed when offloading with optimization compared to when offloading without optimization using the centralized task scheduler.

Zhou et al [19] introduced a technique of offloading that is aware of the contexts for every device to make an offloading decision that relies upon the changes in their contexts in an MCC that is normally heterogeneous. Two algorithms, one online and the other offline, are based on the framework of ski-rental, ensuring optimized performance when offloading and scheduling decisions are taken. This focuses on reducing the time complexity, offloading of the tasks and balancing the loads and also addresses the major issue of the battery limit and enhancement of the offloading. At last, a mechanism which is based on the concept of reverse auction was introduced for the increased utilization and participation of the newly proposed offloading methods in the mobile cloud environment. The results show that there is a notable increase in the efficiency of computation, rationale and auction results that are optimal.

Dong et al [20] presented a comprehensive offloading and resource management framework for embedded systems, which aims to ensure predictable response time performance while minimizing energy consumption. Here developed two offloading algorithms were developed within the framework, which decide the task components that shall be offloaded so that both goals can be achieved simultaneously. This framework was fully implemented on an Android smartphone platform. An in-depth evaluation using representative Android applications and benchmarks demonstrates that our proposed offloading framework dominates existing approaches in terms of timing predictability while effectively reducing energy consumption.

Dou et al [21] presented a framework which provides a powerful software abstraction that hides many of the complexities from the application developer. Design and implement a mobile MapReduce framework targeted at any device which supports Python and network connectivity. The master server node is responsible for assigning tasks to workers when the worker node requests work. The server is also responsible for handling intermediary data generated by the mapping tasks and preparing it for input into the reduce tasks. The implementation of this system is based on a test bed of Nokia N95 8GB smartphones and demonstrates the feasibility and performance.

Elgazzar et al [33] presented a framework for cloud-assisted mobile service provisioning to assist mobile devices in delivering reliable services. The framework supports dynamic offloading based on the resource status of mobile systems and current network conditions, while satisfying the user-defined energy constraints. It also enables the mobile provider to delegate the cloud infrastructure to forward the service response directly to the user when no further processing is required by the provider. Performance evaluation shows up to 6x latency improvement for computation-intensive services that do not require large data transfer.

Gonzalez et al [23] proposed a novel fine-grained application model that supports complex module dependencies such as sequential, conditional, and parallel module executions. The model also allows for multiple offloading decisions that are tailored towards the current application, network, or user contexts. As a result, the model is more precise in capturing the structure of the application and supports more complex offloading solutions. Second, proposed three cost models, namely, average-based, statistics-based and interval-based cost models, defined for the proposed application model. This cost model can be used to perform statistical analysis of the performance of the application in different scenarios, such as varying network data rates. This work provides a robust formulation that solves a linear number of linear optimization problems.

3 Proposed Methodology

The proposed proximate computing framework exploits soft computing methodologies for augmenting the limited resources of mobile devices. The proposed framework consists of five major building blocks, namely: mobile device, context Manager, Resource Augmentation, Quality of Service (QoS) provider and Billing & Cost block, which are depicted in Figure 1. Each block will be explained in a detailed manner. In the mobile device block, the process of the Resource Analyzer & Estimator

component framework is presented. In the context Manager Block, it chooses the appropriate proximate computing entity. The Resource Augmentation block includes a scheduler and synchronizer. The scheduler allocates resources of computing entities between mobile users and their tasks. In the QoS provider block dealt with, three significant parameters, namely security, fault tolerance, and platform independence. Finally billing & cost block is included for cost estimation purposes. Each block will be explained in a detailed manner in the section below. The proposed proximate computing framework for Mobile Resource Augmentation is shown in Fig. 1.

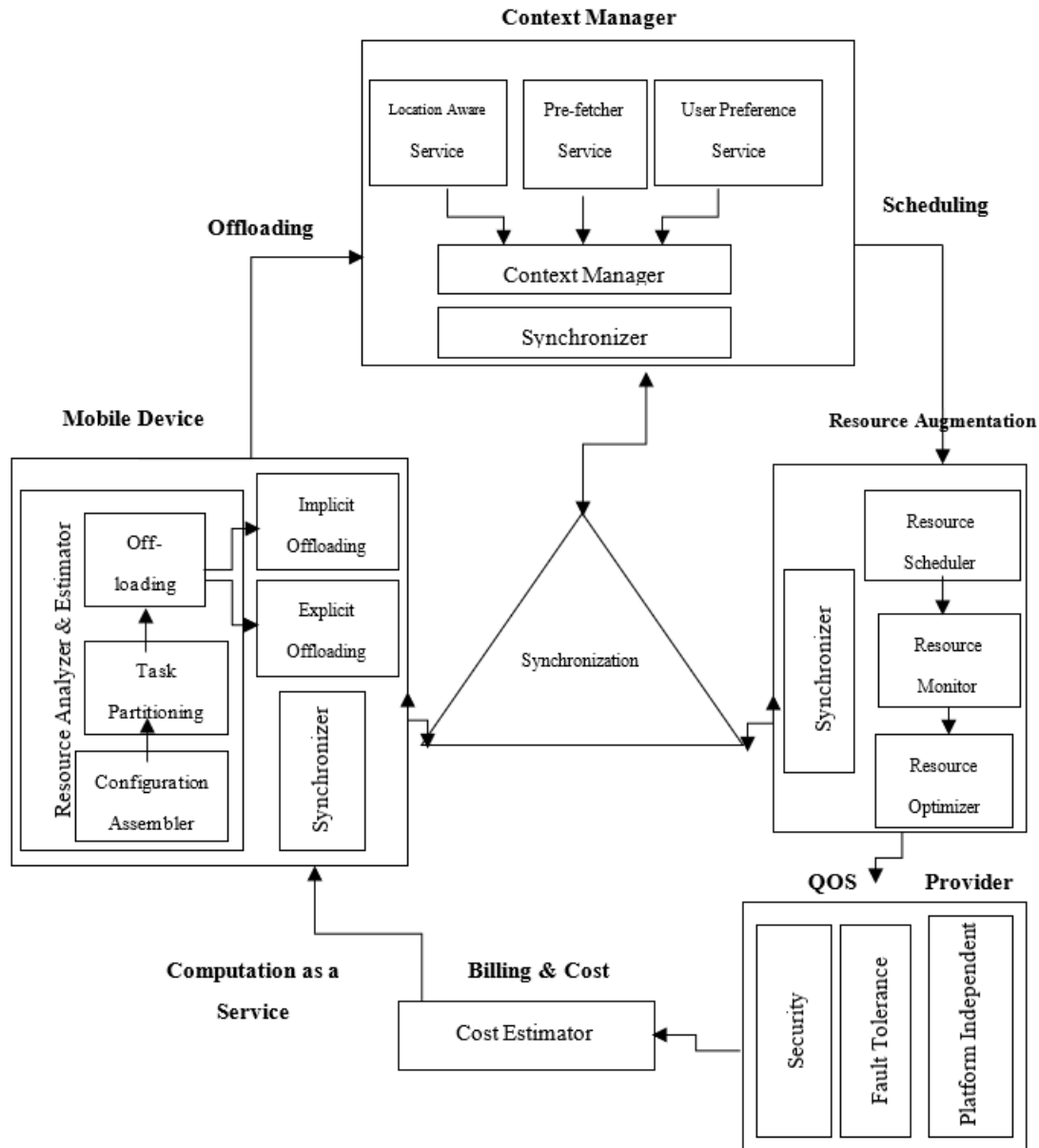


Figure 1: Proximate Computing framework for Mobile Resource Augmentation

3.1 Mobile Device Block

In the mobile device block, the Resource Analyzer & Estimator component consists of three sub-modules, namely configuration assembler, task partitioning, and offloading manager.

3.1.1 Configuration Assembler

The configuration assembler mines parameters about the mobile device, resource-intensive tasks, and network parameters.

3.1.2 Task partitioning

Task partitioning divides resource-intensive tasks into thread-level subtasks, which aid the offloading manager in identifying offload and non-offload subtasks.

3.1.3 Offloading Manager

Offloading is feasible only when local execution time is greater than the offloading time and the remote execution time. Two types of offloading are proposed in the framework, namely i) implicit or automatic offloading and ii) explicit or manual offloading. Implicit offloading restricts the mobile user from exploiting excessive resources in a mobile device by setting a resource-accessing limit or threshold limit. Explicit offloading is performed after getting a response from the mobile user.

Resource Analyzer & Estimator component advocates offloading, the most popular energy-saving technique. As shown in Figure 3, offloading is the process of transferring resource-intensive tasks from the mobile device to proximate computing entities like WiFi Gateway, edge cloud, Raspberry PI controller, Arduino to execute the task and also sends back the result of the task to the mobile device.

3.2 Offloading technique using Deep Neural Network (DNN)

Offloading is a cloud computing technique that is used to run programs and provide content when there a computing-restricted environments. Complex tasks require higher computing power. If the target device has insufficient computing power, the QoS of the provided contents/programs decreases. In this study, the offloading technique is implemented with the help of a Deep Neural Network (DNN) executes a few components of the applications that are in a cloud environment and installed in a mobile device.

Unlike traditional learning, Deep learning is considered a method of learning where the computing systems understand how patterns are recognized and classified from the raw data that is unlabeled(unsupervised) instead of an algorithm that works for a specific task of a general neural network (supervised). The basic futuristic characters lie under various layers that are hidden, and hence, acquiring the name deep learning. These models are intended to make the computers learn independently by identifying the features that decide the character of data using specially designed algorithms. Specific algorithms for sorting and ordering are done as the deep neural networks have various layers that include input, hidden and output layers. This process is termed "Feature Hierarchy".

Deep learning introduces an approach with multiple layers in order to understand the representations of data that are performed with the neural network comprising different layers.

This is similar to other machine learning algorithms, and the learning is achieved by mapping the targets to the features through a process of transformations of data that are simple and by feedback signals. The significance of DNA is that learning takes place in successive layers of representations that bear meaning. The subject is not widely approached owing to its simplicity, but it has proven to be highly successful for the prediction of a variety of problems.

Practically speaking, the neural networks perform tasks that are similar to the human brain. The underlying concept of the neural network has an input, an output, and a hidden layer. The data is read from the inputs given by the user, and the actual learning takes place in the hidden layer, and the output layer projects the results.

Neural networks are simple at the surface level, but the same to be implementing them to see their real working ability requires enormous data for the neural net for learning and also incurs a very high computational cost. This makes the real-time creation of a neural network practically impossible. A great thanks to the development of advanced hardware, such as GPUs, which made the computational tasks more practical and also the rapid growth of data and the collection of the same made it relevant. The development of suitable algorithms helped in removing many constraints to develop a deep neural network [24]. These improvements over time have substantially increased the opportunity to run even very deep and highly parameterized neural networks that are known as deep neural networks (DNNs).

3.3 Feed-forward DNN

These are layers that are connected densely, where all the successful layers are influenced by the inputs and which in turn influence the final output. Deep networks are the normal neural networks which have many layers that are hidden.

The four important components required to construct a feed-forward DNN are

- Input data
- Suitable network architecture
- A defined mechanism for feedback through which learning is achieved.
- A training model

The following section gives a gist of the components to construct a feed-forward DNN for the offloading technique.

3.4 Network architecture

Figure 2 depicts the Architecture of a Feed-Forward DNN. The layers and the activation are the important factors to be considered when developing the architecture of a network.

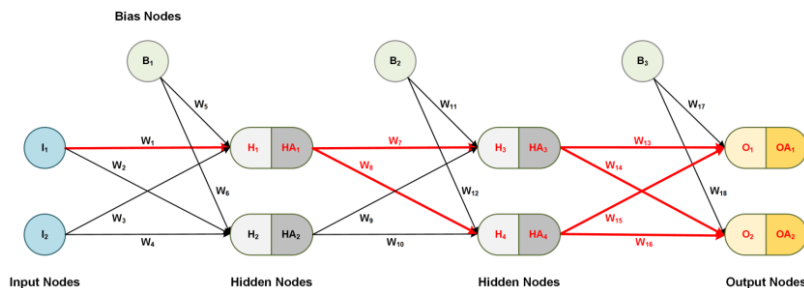


Figure 2: Architecture of Feed-Forward Deep Neural Network

3.5 Layers and nodes

The basic components of a DNN are its layers and nodes, which decide upon how complex the network will be. The condensed layers are those in which all the successful layers are connected. Obviously, when more and more nodes and layers are added, it provides more opportunities for the newly added feature to be learnt (Capacity of the model). Apart from the input layer, which consists of predictor variables, there exist two other layers that are important, namely the hidden layers and the output layer.

3.5.1 Hidden layers

Selecting the hidden layer is vital in selecting the DNN, and the number of nodes deployed in the same is directly responsible for the number of features in the data. Mostly, all the layers have many nodes that are fewer than the number of features. Lastly, the computation overheads are borne by

the number of hidden layers and nodes that are involved. Lastly, the goal is to identify a model which is simple and performs at an optimum level.

3.5.2 Output layers

The model type determines the output layer. Only one node will be present in the output layer for regressive problems, and that node will continuously predict a numeric output. When multinomial output is being predicted, the output layer has the same number of nodes as the classes that are being predicted.

3.6 Activation

The important function of a neural network is its activation. This works similarly to that of biological neurons, where the neurons get constant input and when a threshold value is reached, the same is activated, which shows the existence of a signal. The information flow is given in Figure 3.

3.6.1 Activation functions

All the nodes in a layer are connected with all other nodes in the previous layer [25]. A weight is assigned to each connection and is multiplied by the weight of the corresponding connections with an additional bias. The total of these will be the function.

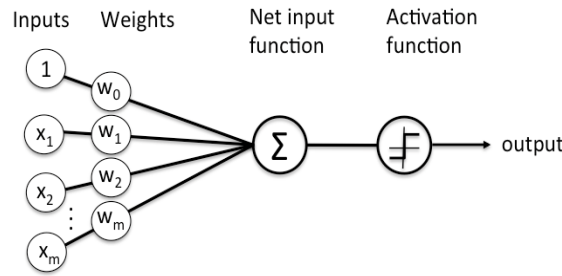


Figure 3: Flow of information in a neuron

These are mathematical functions that identify that there are enough inputs that are informative on a node and trigger signals to the following layer. This requires a determined network structure of layers where each layer is completely connected with the adjacent layer. A common network structure has an input, a hidden and an output layer. The weights are adapted based on the errors calculated in the input patterns, and the technique is feed-forward from the output layer via the input. The function of activation in any given network is denoted by,

$$y_i = \sum_{i=1}^n W_{ij}x_i \quad (1)$$

$$\text{Sigmoid} : f(x) = \frac{1}{1+e^{-x}} \quad (2)$$

The linear activation is done for the outputs that are expected for the problems of regression, and the problems that are of the classification type are taken care of by the sigmoid activation function. The classifications that are multinomial normally use the softmax activation function. The probability provides the optimum solution for the network in a critical state.

Algorithm 1: Feed-forward DNN algorithm for offloading a network model

This training algorithm provides a high-level pseudocode for preparing a network.

Note: It is the base unit for computing denoted as single circle.

Layer: A set of nodes that have a similar index and type.

Connection: A relation that has a weight in between nodes of different layers

W: The connection's weight

I: The node given as input from the neural network

H: Hidden node (The value of all the weights put together of an input layer or the layers that are hidden.

HA: Activated node that is hidden. Hidden node activated (the value of node that is hidden and which is passed to a function that is predefined)

O: The node which comes as an output. (The total weight of the hidden layer taken recently)

OA: Output node activated (The output from the neural network that passes over a function which is predefined.

B: Bias node (A constant value set to 1.0)

e: Total difference between the actual output and the expected value(s) of a network. The errors are computed in terms of metrics such as mean squared error, entropy, etc.

Step 1: Initialization

The primary and initial step after the design of a neural network is its initialization.

- Initialize all weights W_1 through W_{12} with a random number and from a normal distribution, i.e. $\sim N(0, 1)$.

- Set all bias nodes $B_1 = B_2 = 1.0$.

Note: care to be taken to remember that the distribution's variance can be a value that is different cause it affects the convergence.

Step 2: Feed-Forward

As the name indicates, it is performed to go forward and compute all the possible values for any hidden layer considered and the output layer.

- All the input needs are initialized.
- The values of the hidden nodes are calculated.

$$H_1 = I_1W_1 + I_2W_3 + B_1W_5 \quad (3)$$

$$H_2 = I_1W_2 + I_2W_4 + B_1W_6 \quad (4)$$

- The activation function is defined for the hidden layer; for instance, the Sigmoid function:

$$S(x) = \frac{1}{1+e^{-x}} \quad (5)$$

- Calculate hidden node activation values:

$$HA_1 = \frac{1}{1+e^{-H_1}} \quad (6)$$

$$HA_2 = \frac{1}{1+e^{-H_2}} \quad (7)$$

- Calculate output node values:

$$O_1 = HA_1W_7 + HA_2W_9 + B_2W_{11} \quad (8)$$

$$O_2 = HA_1W_8 + HA_2W_{10} + B_2W_{12} \quad (9)$$

- Select an activation function for the output layer; for example, the linear function:

$$y = f(x) = x \quad (10)$$

- Calculate output node activation values:

$$OA_1 = O_1 \quad (11)$$

$$OA_2 = O_2 \quad (12)$$

- Calculate the total error; if OA_i is the obtained output value for node i , then let y_i be the desired output

$$e = \frac{1}{n} \sum_{i=1}^4 (y_i - OA_i)^2 \quad (13)$$

After the first pass, the error will be substantial, but we can use an algorithm called backpropagation to adjust the weights to reduce the error between the output of the network and the desired values.

Step 3: Backpropagation

When the model is run for the first time, or in a forward pass, A series of observations is recorded by the DNN that assigns weights randomly between all the connections of a node and the prediction of output is done. Back Propagation is termed as the process by which a neural network, by itself, assesses its performance and adjusts the weights between all the connections of the node to improve the accuracy. The objective of this is to adjust the weights in an incremental fashion to enable the networks to come up with values that are very close when compared to the expected values obtained from the training data.

Backpropagation is used for adjusting the weight of the network with the formula,

$$W_i^{k+1} = W_i^k - \eta \frac{\partial e}{\partial W_i^k} \quad (14)$$

Where k is the iteration number, η is the learning rate (typically a small number), and: $\frac{\partial e}{\partial W_i^k}$ is the total error's derivation with respect to the adjusted weight. In this process, a formula is derived for every individual weight present in a network, which also includes the weights allotted for connections.

Step 4: Model training

Once a base model is created, Training is to be given from the available data. To achieve this, the model is fed into a fit function in addition to the training data. The model which is optimum is to be found for producing and running the DNN model. These are done using various parameters, including the capacity of the model, normalization of the batch, dropout and regularization weight and learning rate.

$$Y = F(y_{in}) \quad (15)$$

Based on the Y value, execution of the subtask is decided. After applying the activation function in the training algorithm, based on the output value of Y, the subtask is executed in three different ways, namely local execution in the mobile device itself (Y value < 0.5), implicit offloading (Y value is between 0.5 & 1) and explicit offloading (Y value > 1). As depicted in Figure 4, the proposed system exploits the adaptive reinforcement learning neural network to choose the offloading method based on the input values from the resource analyzer and estimator. In an adaptive neural network, the overall input-output behaviour is determined by a collection of modifiable parameters from the configuration assembler (x1) and task partitioning (x2). Under reinforcement learning, supervised learning advocates explicit offloading and unsupervised learning promotes implicit offloading.

$$Y = \begin{cases} < 0.5 & \text{local execution} \\ \geq 0.5 \text{ and } \leq 1 & \text{implicit offloading} \\ > 1 & \text{explicit offloading} \end{cases} \quad (16)$$

3.7 Context Manager Block

In this block, the context Manager chooses the appropriate proximate computing entity. After deciding offloading mode, the Context manager exploits a Genetic Algorithm to select the optimal proximate computing entity. Context Manager chooses the appropriate proximate computing entity based on the input from its three subcomponents, namely a) location-aware service, b) pre-fetcher service and c) user preference service. The Fig.4. shows the selection of a Proximate computing entity.

3.7.1 Location Aware Service (LAS)

The location-based service provides validation with reference to the different networking services. This is done to identify the efficiency of the location-based service to access various networking services. Location-aware service (LAS) provides the following attributes of the mobile device and Proximate Computing Entity (PCE) by exploiting the NearBy connection API.

$$LAS = \{c_{mlon}, c_{mlat}, pce_{tsize}, pce_{texetime}, pce_{tenecon}\} \quad (17)$$

Where

c_{mlon}	- current longitude of the mobile device
c_{mlat}	- current latitude of the mobile device
pce_{tsize}	- size of the task
$pce_{texetime}$	- execution of the time of the task
$pce_{tenecon}$	- energy consumed by the task

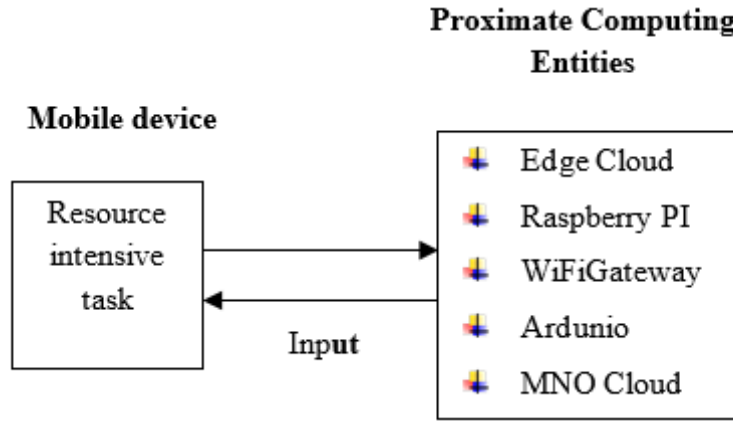


Figure 4: Selecting a Proximate computing entity

3.7.2 Pre-Fetcher Service (PFS)

Different wireless access technologies like Wi-Fi/3G/4G exploit different amounts of energy and support different data transfer rates. Since the location of the mobile user is not fixed, offloading must ensure retransmission of lost packets due to network disconnection or lower signal strength. The functionality of Pre-fetcher is to assist the context manager with proximate computing entity information whenever the user changes their location. Depending on the user's location, all updated content will be pre-fetched and cached. All cached content will be notified to the context manager, and this content will be post-fetched if required with respect to the location. Pre-fetch service (PFS) provides the updated attributes of the mobile location and proximate computing entity (PCE).

$$PFS = \{n_{mlon}, n_{mlat}, npce_{tsize}, npce_{texetime}, npce_{texecost}, npce_{tenecon}\} \quad (18)$$

Where

n_{mlon}	- new longitude of the mobile device
n_{mlat}	- new latitude of the mobile device
$npce_{tsize}$	- size of the task
$npce_{texetime}$	- the execution time of the task
$npce_{tenecon}$	- energy consumed by the task

3.7.3 User Preference Service (UPS)

User preference modes such as power saving mode, low-cost mode, high processing mode, and high storage mode have played a vital role in the selection of a proximate computing entity. For example, low-cost mode exploits the computing entity without any Internet connection. High processing mode advocates faster response to the mobile user by exploiting a resource-rich computing entity. User preference mode is expressed as an XML document and is depicted below.

```

<?xml version="1.0" encoding="UTF-8"?>
<user_preference_mode>
  <power_saving offload="yes"> battery_capacity </power_saving>
  <low_cost offload="yes|no" Internet_con="no" cloud_service="no">
    execution_cost
  </low_cost>
</user_preference_mode>
  
```

```

        </low_cost>
        <high_proc offloading="yes" cloud_service="yes|no">
            response_time
        </high_proc>
        <high_storage offloading="yes" cloud_service="yes">
            storage_space
        </high_storage>
    </user_preference_mode>

```

Algorithm 2: Pseudo Code of a genetic algorithm to find an optimal proximate computing entity.

BEGIN

List the existing proximate computing entities;

Compute the fitness value of each computing entity;

WHILE NOT finished DO LOOP

BEGIN

Selection;

Apply ordinal based **Rank Selection** method to elect the best computing entities from the existing pool;

Reproduction;

Apply **whole arithmetic recombination crossover** and optional **Random resetting Mutation;**

Evolution;

Compute the **fitness value** of new computing entity;

Replacement;

Replace old computing entity with a new one;

Termination;

IF existing list has converged **THEN**

finished : = TRUE;

ENDIF;

END;

END;

In the genetic algorithm, the rank selection is applied to select the best proximate computing entity. Moreover, rank selection advocates computing entities with closer fitness values. Every computing entity is ranked according to its fitness values. Higher-ranked computing entity is preferred over a lower-ranked computing entity. Fitness function F is implemented as an inverse objective function of input parameters from the Location-aware service, Pre-fetcher service and User preference service.

$$F = \frac{1}{(ls+ps+up-t)} \quad (19)$$

Where ls is a location-aware service parameter, ps is a pre-fetcher service parameter, up is a user preference parameter, and t is a threshold value.

3.8 Resource augmentation block

The Resource Augmentation block includes a scheduler and synchronizer. The scheduler allocates resources of computing entities between mobile users and their tasks. This work focused on three different types of services: native execution, proximate or edge cloud and Arduino computing services. Consider the following list, which consists of n proximate computing entities.

$$pce_{lst} = \{pce_1, pce_2, pce_3 \dots pce_n\} \quad (20)$$

Consider a task t that consists of m subtasks.

$$t = \{st_1, st_2, st_3 \dots st_m\} \quad (21)$$

3.9 Task Scheduling

After selecting the appropriate offloading method from the Deep Neural Network and proximate computing entity from the genetic algorithm proposed system exploits Bat optimization-based Resource Optimizer is introduced which optimizes the resource constraints of mobile devices. For optimizing the resources of mobile devices, a model is performed based on a centralized broker-node to schedule subtasks in the selected proximate computing entity.

Since the Task Scheduling plays a significant role in improving the flexibility and reliability in the Resource Augmentation block. The ultimate reason behind scheduling tasks for resource augmentation involves the technique of finding the best sequence in several tasks. The task scheduling selects the appropriate resource which is available for task execution in an effective manner, in which completion time and battery power are reduced as far as possible. The sub-tasks are selected as per the priorities, as well as assigned to the selected proximate computing entity.

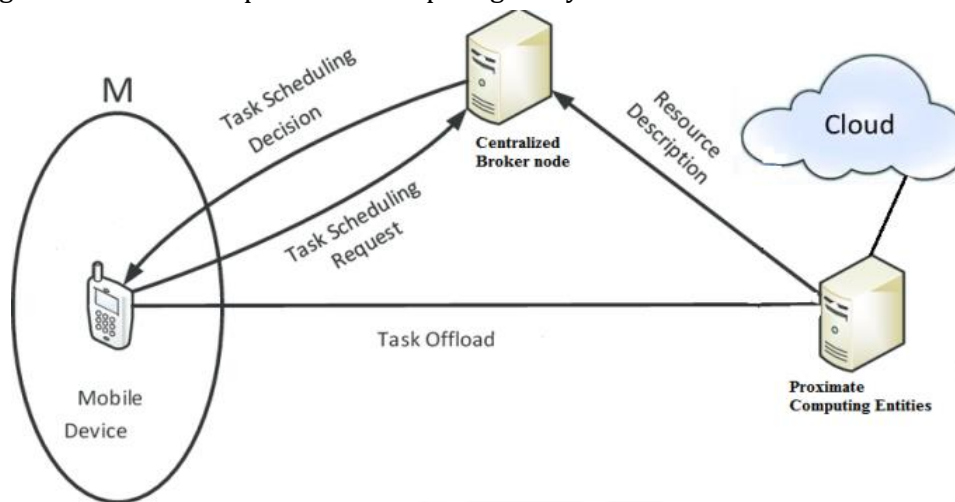


Figure 5: Centralized broker node model for resource Augmentation

The proposed centralized broker model is employed for the resource augmentation of mobile devices. Though the proposed work extends the optimization process by including the Bat optimization [26] algorithm. The model presented in this work is enhanced to make it more realistic. This proposed work is modelled using the BAT optimization algorithm with adjustable parameters to schedule the sub-tasks.

On behalf of all the mobile devices, the centralized node gathers resource descriptions from all the service nodes in the system. The centralized node consolidates all the resource descriptions and uses this information when scheduling task offloading on behalf of all mobile devices in the system. Thus, the centralized node can find the best available resources for all the mobile devices based on their requirements and offloading goals. In this way, the mobile devices need not repeatedly contact all the service nodes for resource monitoring. The centralized node was referred to as broker node. Also, in this work, the location of the task scheduler is the centralized broker-node, as shown in Figure 5. The various steps involved while scheduling tasks from mobile devices onto the resources in the selected proximate computing entity which include:

1. Mobile devices contact the centralized task scheduling service at the broker-node,
2. The task scheduler decides the appropriate offloading location on behalf of the mobile devices by minimizing the total energy consumption and the total monetary cost across all mobile devices subject to various constraints, and
3. Based on the task scheduling decision, each task is either offloaded or it is executed locally on the mobile device. Using the broker-node, the communication between the two scenarios differs at: (i) the infrastructural level, (ii) the location of the TaskScheduler() service, and (iii) the mechanism for the resource monitoring process is depicted in Figure 6.
4. At the infrastructural level, the centralized broker model includes a centralized broker-node.
5. The TaskScheduler() service that manages task offloading is utilized on the mobile devices.
6. On behalf of all the mobile devices using the ResourceMonitor() service, the RB Client() service in the mobile devices can get resource descriptions of all the service nodes just by contacting only the ServerApp() service in the broker-node.
7. The centralized broker-node also uses the ServerApp() service to accept resource description requests from the RB client() service in the mobile devices.

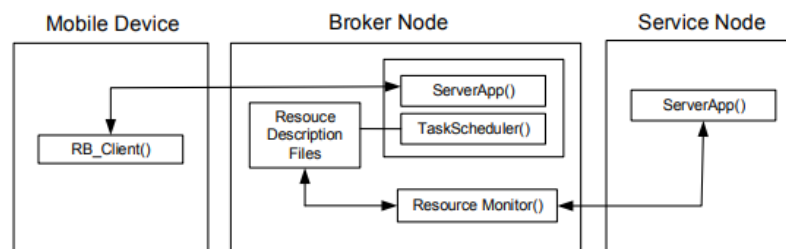


Figure 6: Centralized broker scenario.

Algorithm 3: Priority-based Bat Optimisation algorithm to execute the subtask.

Input : 'm' number of subtasks and a single proximate computing entity (pce)

Output : finding priority order and execution of the subtask.

Assign no. of tasks $f(x)$, $X = (x_1, x_2, \dots, x_d)^T$

Initialize sub tasks x_i ($i = 1, 2, \dots, n$) and v_i

Define priority order f_i at x_i


```

Initialize computing entity  $r_i$  and execution of sub task  $A_i$ 
While (  $t < \text{Max number of iterations}$ )
Generate new solutions by adjusting priority, and
Updating priority order and locations/solutions
    If (  $\text{rand} > r_i$ )
        Select a solution among the best solutions
        Generate a local solution around the selected best solution
    end if
    Generate a new solution by selecting subtask randomly
    if (  $\text{rand} < A_i \ \& \ f(x_i) < f(x.)$ )
Accept the new solutions
    Increase  $r_i$  and reduce  $A_i$ 
    end if
Rank the priority and find the best subtask  $x$ .
end while
Results / visualizations

```

Algorithm 4: Pseudo-code of the proposed algorithm

```

Step 1: for each task T consider User's Requests
Step 2: Split T into Sub_tasks ( $T_i$ )
Step 3: Call Bat_algo(Sub_task)
Step 4: Iterate (Repeat Bat_algo until Best_sol)
Step 5: end;
Step 6: Apply Column Maxima to Best_sol
Step 7: Schedule thus obtained is the input to Computing entity

```

3.10 Task Scheduling

And the proposed system, three blocks: mobile device, context manager and resource augmentation, has contained synchronizer component. Figure 7 depicts the functionality of the synchronizer, which facilitates the faster updating of data between the mobile device and proximate computing entities.

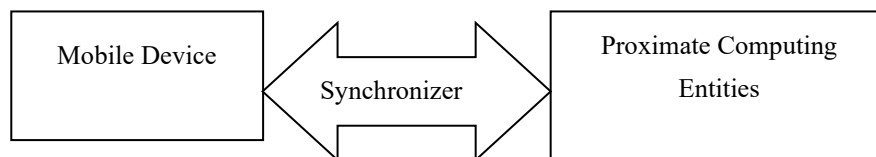


Figure 7: Synchronization

3.11 Quality of service (Qos) provider block

Three significant parameters, namely security, fault tolerance and platform independence, are dealt with in the QOS provider block. Exploitation of fewer Internet or no Internet connectivity ensures security

in the proposed framework. In case of failure in the current proximate computing entity, fault tolerance ensures the availability of computation as a service from another proximate computing entity. Platform-independent feature authorizes mobile devices with any mobile operating system, such as Android, BlackBerry, Windows Mobile and iOS to access the computing services. The proposed framework augments the resources of the mobile device by providing computation as a service to the mobile device.

3.12 Execution steps for the proposed framework

The flow of operations to augment the limited resources of mobile devices in the proposed proximate computing framework is depicted in Figure 8. A requisition for augmenting mobile resources is initiated after the submission of the task by the mobile user. Now, mobile resource augmentation is done by providing computation as a service to the mobile user or the task is executed locally as per the internal operations of the proposed framework.

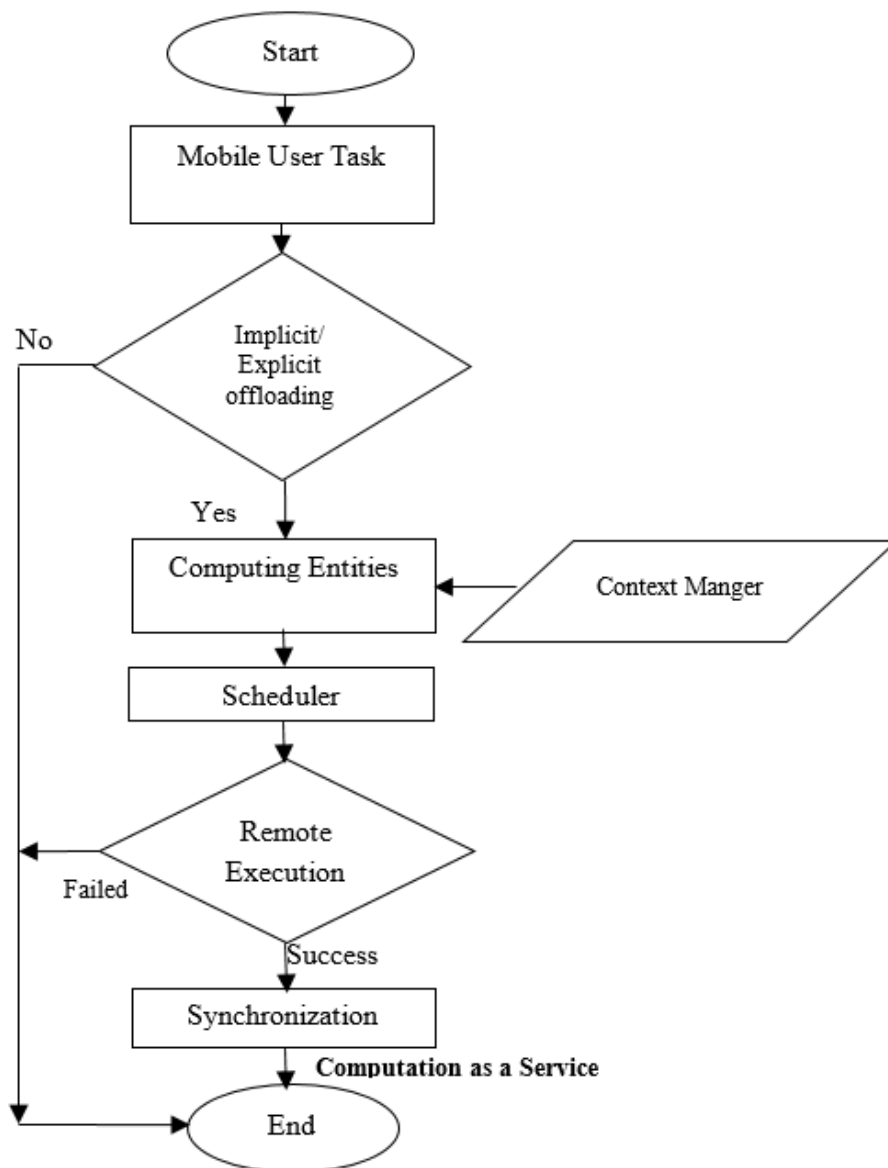


Figure 8: Flowchart for Proposed Framework Execution

Algorithm 5: Execution of Proposed Framework and Pseudo Code of Building Block Operations

```

Begin;
for each mobile user request do
  if offloader > 0 then
    Fetch (computing entities from context manager) ;
    Scheduler (from resource augmentation);
    QOS parameters (from QOS provider)
    if execution succeed then
      Synchronize data;
      Computation as a service;
    else
      Handle Error; retry or terminate execution;
  else
    Local Execution;
  Execution Finished;
End.

```

3.13 Intelligent Inventory Checker

In this inventory checker Mobile Application, two proximate computing entities, namely Arduino and edge cloud, provided computation as a service by executing the resource-intensive task of checking reorder level. In supermarkets, the supervisor needs to do regular reviews of product availability. If there is any shortage, then the supervisor must place an order to sustain stocks to a predetermined level. This task necessitates more processing time and energy consumption and may lead to customer dissatisfaction and customer loss. So, this resource-intensive task of checking reorder level is offloaded to a proximate computing entity, namely Arduino, edge cloud, thereby providing a seamless user experience to the customer. Therefore, an intelligent inventory checker mobile application is automated and executed in three different methods, namely Native Mobile App, Proximate Cloud and Arduino application.

Task: Inventory Checker

Method 1: Native Mobile App (Android OS)

Algorithm 6:

Step 1: Instantiate a Sensor Manager Object.

Step 2: Registers a Sensor Event Listener to read the raw data of the proximity sensor for every five seconds.

Step 3: Inside get Maximum Range() method of Sensor object, Calculate stock value.

Step 4: Display the inventory details such as item id, item name, the image of the item on mobile.

Step 5: Find the execution time of the code by invoking System. Current Time Millis() method and energy consumed by using Power tutor.

The Native MobileApp for checking the reorder level of the stock is depicted in Figure 9.

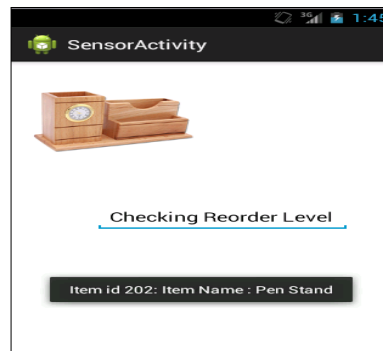


Figure 9: Intelligent Inventory Checker Mobile App

Method 2: Proximate Cloud

As shown in Figure 10, the proposed IoT application using IBM Bluemix sends SMS and email notifications to the supervisor whenever the size of stock is reduced to the reorder level. Node-RED is a programming tool for connecting hardware devices, APIs and online services. This instance is running as an IBM Bluemix application, giving it access to the wide range of services available on the platform.

Algorithm 7:

- Step 1: Create an IBM Bluemix account.
- Step 2: Enter into Node-RED flow editor.
- Step 3: Design a flow with required nodes.
- Step 4: Deploy it and test with JSON input.
- Step 5: Display the inventory details such as item id, item name, the image of the item.

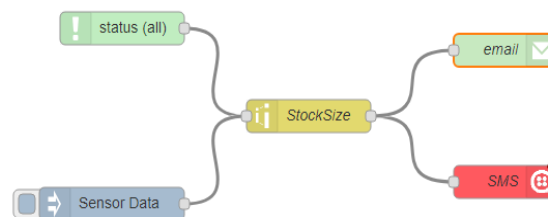


Figure 10: Node-RED flow design for Inventory Checker

Method 3: Arduino application

The proposed system automates an intelligent inventory checker with an Arduino and an Intranet/Internet web programming to find the shortage, which is indicated by an automated SMS process to help maintain adequate stock. The architecture of the Inventory Checker application is depicted in Figure 11. The proposed system consists of the following four modules, namely the intelligent stock checker, the Stock monitor, SMS integration, and Cloud deployment.

Intelligent stock checker: This Arduino-based intelligent stock checker alerts the supervisor by blinking the LED when the stock reaches the preset level. This circuit needs to connect to the server to make an alert about the stocks. This stock checker Mobile App reduces the time to verify all the stocks in the store.

Stock monitor: Once the tray is empty or the product count starts reducing, an alert will be sent to the supervisor. This can be controlled by a web interface, which also monitors the availability of stocks.

SMS integration: The stock value will be notified to the supervisor through an SMS API.

Cloud deployment: This web application is deployed in a Bluemix cloud environment, which offers a broad set of global cloud-based products including computer storage, databases, analytics, networking, mobile, developer tools, management tools, IOT, security and enterprise applications.

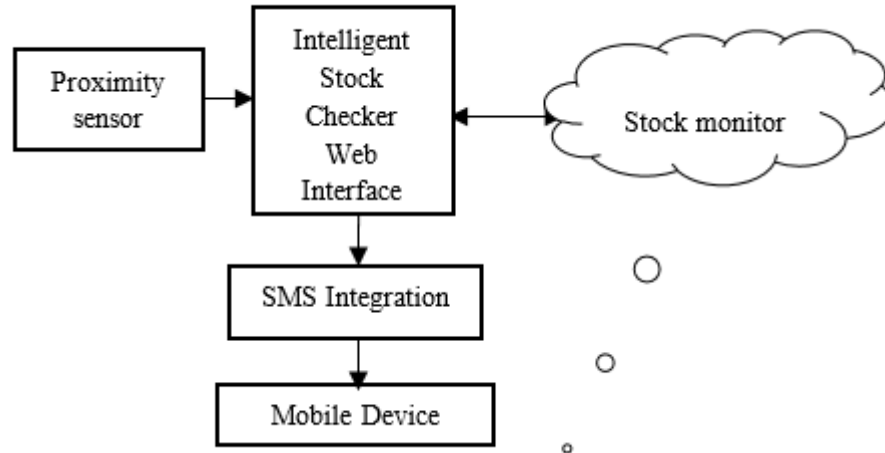


Figure 11: Architecture Diagram of Inventory CheckerApp

3.14 Testing Arduino LED

To build the circuit, a 220-ohm resistor is attached to pin 13. The long leg of an LED is connected to the resistor, and the short leg to ground. The Arduino board is plugged into the system, and the Arduino program is executed. Most Arduino boards already have an LED attached to pin 13 on the board itself. LED blinks to show the reorder level of stock. Figure 12 shows the reorder checking alerts to the supervisor by blinking the LED.

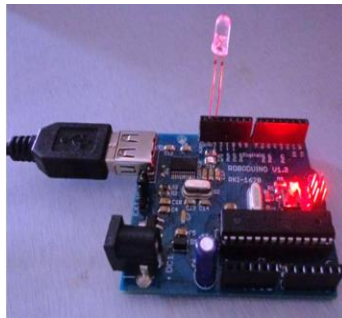


Figure 12: Testing Arduino-LED Blink

4 Results and Discussion

An Intelligent inventory Checker application is implemented and analyzed with three proximate computing entities with various comparison parameters.

Table 1: Execution time (ms) of computing entities

Computing Entities	Existing Proximate Computing framework using NN				Proposed Proximate Computing framework using DNN			
	Exec 1	Exec 2	Exec 3	Exec 4	Exec 1	Exec 2	Exec 3	Exec 4
NativeApp	458	346	645	568	455	343	641	566

Bluemix App	345	243	542	455	342	240	539	453
Arduino App	312	212	457	356	308	207	454	355

Table 2: Energy consumption (MW) of computing entities

Computing Entities	Existing Proximate Computing framework using NN				Proposed Proximate Computing framework using DNN			
	Exec 1	Exec 2	Exec 3	Exec 4	Exec 1	Exec 2	Exec 3	Exec 4
NativeApp	410	294	556	524	406	291	551	520
Bluemix App	305	205	498	405	301	200	495	404
Arduino App	276	198	402	310	272	194	399	306

Based on the execution time and energy consumption of three computing entities, which are shown in Tables 1 and 2, comparison parameters of the three computing entities are depicted in Table 3.

Table 3: Comparison of computing entities

Parameter	Native MobileApp	IBM Bluemix App	Arduino App
Execution Time	High	Medium	Low
Energy Consumption	High	Medium	Low
Communication Cost	Low	Medium	Medium
Computation Cost	Low	Medium	High

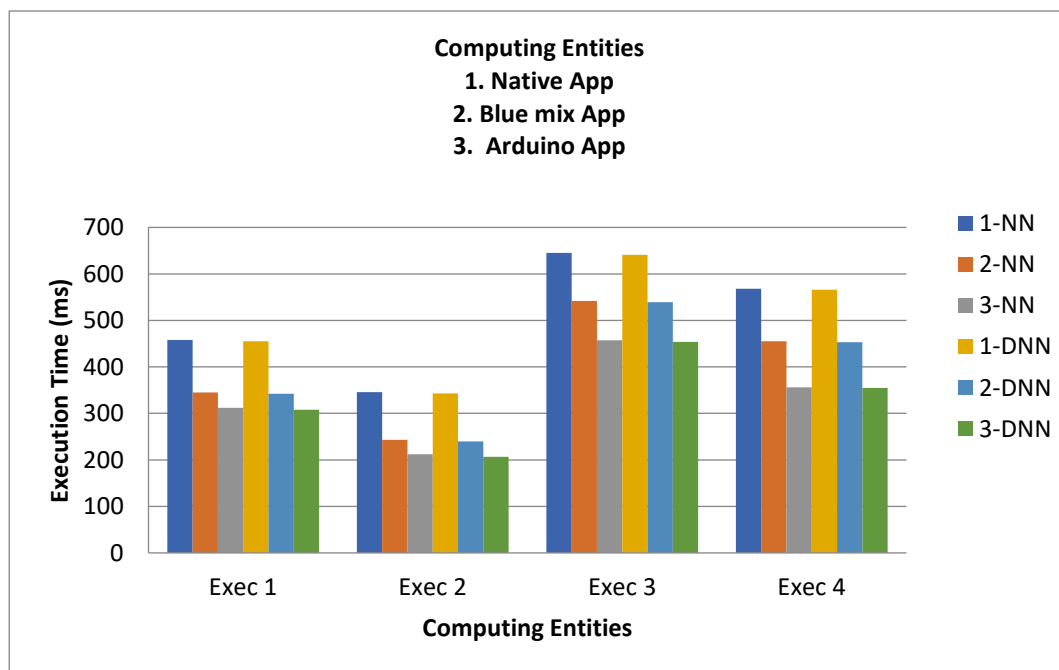
**Figure 13:** Execution Time of computing entities

Table 3 depicts the significance of proximate computing entities over the native Mobile App. Proximate computation as a service mitigates execution time and energy consumption, thereby augmenting the resources of mobile devices. However, the native Mobile App is cost-effective in terms of communication and computation. Moreover, no additional hardware or Internet connectivity is required for the execution of a native mobile app. The charts in Figures 13 and 14 show that the native mobile application requires more energy and execution time than the other proximate computing entities. Therefore, it is trustworthy to provide computation as a service to augment the resources of mobile devices. However, execution of the native mobile application is preferred while executing a cost-effective application.

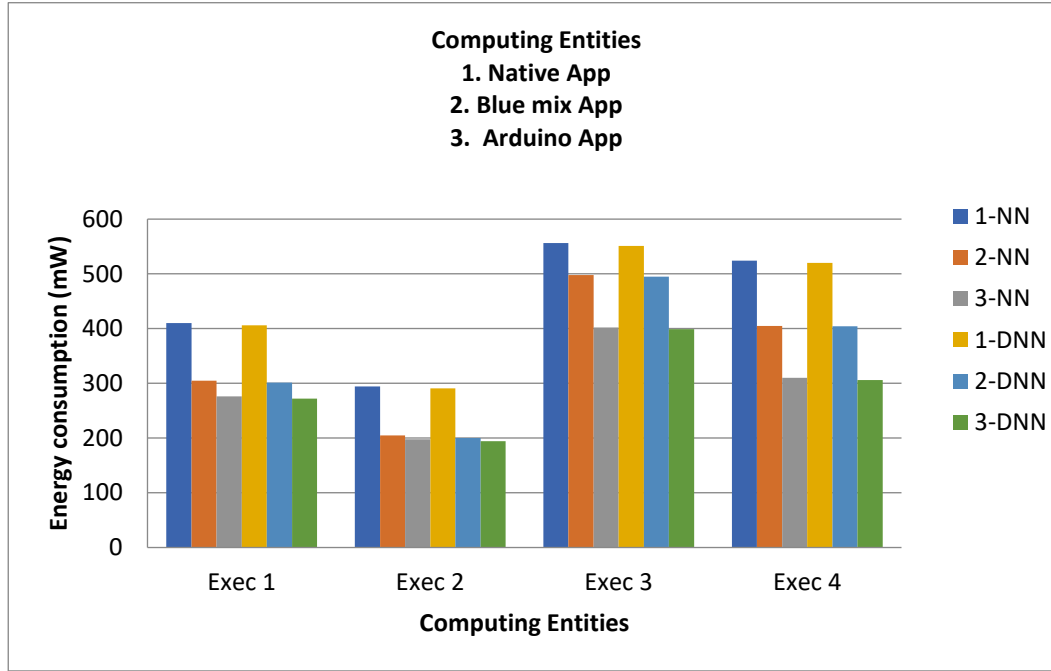


Figure 14: Energy Consumption of computing entities

In this proposed framework, both execution time (Exe_{time}) in milliseconds and energy consumption (Ene_{cons}) in milliwatts are measured in the same method, as shown below.

For Android Mobile App

$$T_{Exe_{time}} = S_{Exe_{time}} + M_{Exe_{time}} \quad (22)$$

$$T_{Ene_{cons}} = S_{Ene_{cons}} + M_{Ene_{cons}} \quad (23)$$

Where

$T_{Exe_{time}}$ = Total Execution Time

$S_{Exe_{time}}$ = Execution Time to read the proximity sensor data to the mobile application

$M_{Exe_{time}}$ = Execution Time to check reorder level with mobileApp.

For the IBM Bluemix Application

$$T_{Exe_{time}} = N_{Exe_{time}} + J_{Exe_{time}} \quad (24)$$

$$T_{Ene_{cons}} = N_{Ene_{cons}} + J_{Ene_{cons}} \quad (25)$$

Where

$T_{Exe_{time}}$ = Total Execution Time

$N_{Exe_{time}}$ = Execution Time to design and deploy the Node-RED flow system

$J_{Exe_{time}}$ = Execution Time to check reorder level with JSON input.

For Arduino Application

$$T_{Exe_{time}} = S_{Exe_{time}} + W_{Exe_{time}} + C_{Exe_{time}} \quad (26)$$

$$T_{Ene_{cons}} = S_{Ene_{cons}} + W_{Ene_{cons}} + C_{Ene_{cons}} \quad (27)$$

Where

$T_{Exe_{time}}$ = Total Execution Time

$S_{Exe_{time}}$ = Execution Time to read sensor data

$W_{Exe_{time}}$ = Execution Time for web interface

$C_{Exe_{time}}$ = Cloud Deployment time

Table 4: Total Energy consumption (mw) when offloading with and without optimization in selected computing entities

Computing Entities	Offloading	Offloading with NN	Offloading with DNN
NativeApp	621	553	524
Bluemix App	480	431	405
Arduino App	364	324	310

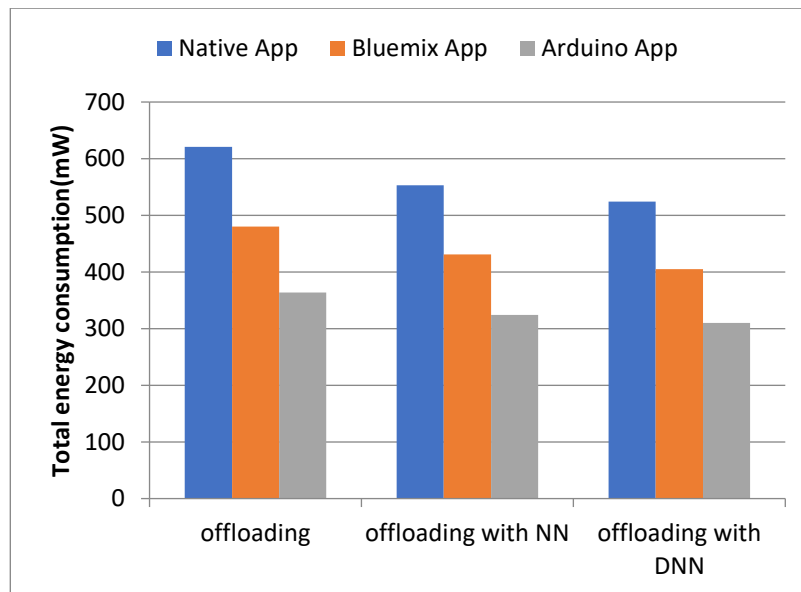


Figure 15: Total Energy Consumption when offloading with and without optimization in selected computing entities

The total energy consumption when offloading with and without optimization in selected computing entities was analyzed in fig.15. It concludes that the proposed offloading with the DNN computing framework consumes less power compared to the existing offloading with the NN computing framework.

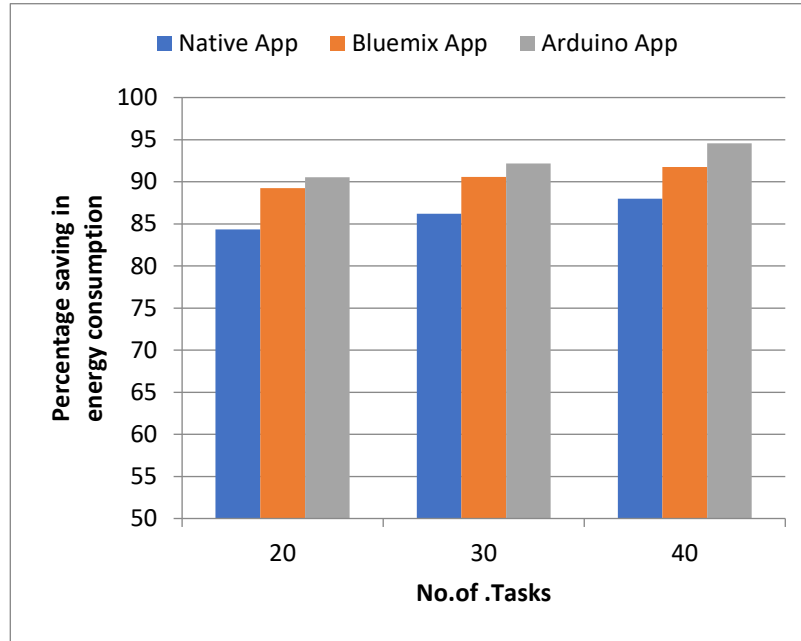


Figure 16: Percentage saving in Total Energy Consumption while increasing the no. of tasks in selected computing entities

Fig. 16 shows the Percentage saving in Total Energy Consumption while increasing the no. of tasks in selected computing entities. It concludes that the Arduino App has more Percentage saving in the range of 94.56%, while increasing the number of tasks (40) as compare to the Bluemix App has less Percentage saving in the range of 91.75% and the Native App has less Percentage saving in the range of 88%.

5 Conclusion and Future Work

Mobile cloud computing has emerged as a software-level solution to augment the computing power of resource-scarce mobile devices. The open issues like Energy consumption, cost model, service quality assurance, security, and standardized architecture need to be dealt with as prime considerations for the further success of mobile cloud computing. The proposed framework is to utilize a centralized broker-node to perform task scheduling for the resource augmentation of a large number of mobile devices. The task scheduler model focused on bat optimization-based Resource Optimizer was proposed which optimizes the resource constraints of mobile devices. This work also proposed a novel offloading algorithm called Deep Neural Network (DNN), which performed as many tasks as possible in the cloud when the network transmission bandwidth is high, thereby improving the total execution time of all tasks and minimizing the energy use of the mobile device. In this work, the proximate computing entities are used to augment the resources of the mobile device. In this research work, Arduino and Edge Cloud services provide computation as a service to augment the processing time and energy consumption of the mobile device. An intelligent inventory checker mobile application based on the proposed framework depicts significant mitigation in execution time and energy consumption of mobile devices. This proposed model is evaluated under two different resource augmentation environments for mobile cloud computing: More precisely, the task

scheduling problem is optimally solved to minimize: (i) the total energy consumption and (ii) the execution time. In future enhancements, like a power bank, a processor bank will be implemented as a part of computation as a service to augment the resources of mobile devices. Also, there is a plan to implement IoT-based MNO cloud services by exploiting fog computing. The cost estimator module of the proposed system will be implemented by applying bidding schemes.

Acknowledgement: Not applicable.

Funding Statement: Not applicable.

Author Contributions: The authors confirm contribution to the paper as follows: Conceptualization, methodology and writing—original draft preparation: M. Nalini; supervision and writing—review and editing: Sivakumar Vengusamy. All authors reviewed the results and approved the final version of the manuscript.

Conflicts of Interest: The authors declare no conflicts of interest to report regarding the present study.

References

1. Dimou A, Iliopoulos C, Polytidou E, Dhurandher SK, Papadimitriou G, Nicopolitidis P. A comprehensive review on edge computing: Focusing on mobile users. *Advances in Computing, Informatics, Networking and Cybersecurity: A Book Honoring Professor Mohammad S. Obaidat's Significant Scientific Contributions*. 2022;121-52.
2. Noor TH, Zeadally S, Alfazi A, Sheng QZ. Mobile cloud computing: Challenges and future research directions. *J Netw Comput Appl*. 2018;115:70-85.
3. Peñalvo FJ, Sharma A, Chhabra A, Singh SK, Kumar S, Arya V, Gaurav A. Mobile cloud computing and sustainable development: Opportunities, challenges, and future directions. *Int J Cloud Appl Comput*. 2022;12(1):1-20.
4. Shah Z, Javed U, Naeem M, Zeadally S, Ejaz W. Mobile edge computing (MEC)-enabled UAV placement and computation efficiency maximization in disaster scenario. *IEEE Trans Veh Technol*. 2023;72(10):13406-16.
5. Thabet S, Ateya AA, Abo-Zahhad M. Mobile edge computing-based radio access network for tactile internet. In *2021 9th International Japan-Africa Conference on Electronics, Communications, and Computations (JAC-ECC)*, 2021 Dec 13, Alexandria, Egypt, 2021, p. 119-124.
6. Zhu M, Gao S, Tu G, Chen D. Multi-access edge computing (MEC) based on MIMO: A survey. *Sensors*. 2023;23(8):3883.
7. Haibeh LA, Yagoub MC, Jarray A. A survey on mobile edge computing infrastructure: Design, resource management, and optimization approaches. *IEEE Access*. 2022;10:27591-610.
8. Chen X, Li M, Zhong H, Chen X, Ma Y, Hsu CH. FUNOff: Offloading applications at function granularity for mobile edge computing. *IEEE Trans Mob Comput*. 2023;23(2):1717-34.
9. Hosseinpour M, Moghaddam MH. Quality-of-experience-aware computation offloading in MEC-enabled blockchain-based IoT networks. *IEEE Internet Things J*. 2023;11(8):14483-93.
10. Kovachev D, Yu T, Klamma R. Adaptive computation offloading from mobile devices into the cloud. In *2012 IEEE 10th International Symposium on Parallel and Distributed Processing with Applications*, 2012 Jul 10, Leganes, Spain, 2012, p. 784-791.
11. Kemp R, Palmer N, Kielmann T, Bal H. Cuckoo: a computation offloading framework for smartphones. In *International Conference on Mobile Computing, Applications, and Services*, 2010 Oct 25, Berlin, Heidelberg, 2010, p. 59-79.
12. Orsini G, Bade D, Lamersdorf W. CloudAware: empowering context-aware self-adaptation for mobile applications. *Trans Emerg Telecommun Technol*. 2018;29(4):e3210.
13. Quoc PC, Nguyen XQ, Thinh TN. Towards an FPGA-targeted hardware/software co-design framework for CNN-based edge computing. *Mob Netw Appl*. 2022;27(5):2024-35.
14. Shiraz M, Gani A, Shamim A, Khan S, Ahmad RW. Energy efficient computational offloading framework for mobile cloud computing. *J Grid Comput*. 2015;13(1):1-8.

15. Deng Y, Chen Z, Yao X, Hassan S, Ibrahim AM. Parallel offloading in green and sustainable mobile edge computing for delay-constrained IoT system. *IEEE Trans Veh Technol.* 2019;68(12):12202-14.
16. Shiraz M, Gani A. A lightweight active service migration framework for computational offloading in mobile cloud computing. *J Supercomput.* 2014;68(2):978-95.
17. Kumar J, Malik A. Contextual Computation Augmentation in Mobile Cloud Computing. *Int J Adv Res Comput Sci.* 2018;9(2):65.
18. Nir M, Matrawy A, St-Hilaire M. Economic and energy considerations for resource augmentation in mobile cloud computing. *IEEE Trans Cloud Comput.* 2015;6(1):99-113.
19. Zhou B, Dastjerdi AV, Calheiros RN, Srirama SN, Buyya R. mCloud: A context-aware offloading framework for heterogeneous mobile cloud. *IEEE Trans Serv Comput.* 2015;10(5):797-810.
20. Dong Z, Liu Y, Zhou H, Xiao X, Gu Y, Zhang L, Liu C. An energy-efficient offloading framework with predictable temporal correctness. In *Proceedings of the Second ACM/IEEE Symposium on Edge Computing*, 2017 Oct 12, New York, NY, USA, 2017, p. 1-12.
21. Dou A, Kalogeraki V, Gunopulos D, Mielikainen T, Tuulos VH. Misco: a mapreduce framework for mobile systems. In *Proceedings of the 3rd international conference on pervasive technologies related to assistive environments*, 2010 Jun 23, New York, NY, USA, 2010, p. 1-8.
22. Elgazzar K, Martin P, Hassanein HS. Cloud-assisted computation offloading to support mobile services. *IEEE Trans Cloud Comput.* 2014;4(3):279-92.
23. Gonzalez M, López-Espín JJ, Aparicio J, Talbi EG. A hyper-matheuristic approach for solving mixed integer linear optimization models in the context of data envelopment analysis. *PeerJ Comput Sci.* 2022;8:e828.
24. Chen X, Li M, Zhong H, Ma Y, Hsu CH. DNNOff: Offloading DNN-based intelligent IoT applications in mobile edge computing. *IEEE Trans Ind Inform.* 2021;18(4):2820-9.
25. Cil AE, Yildiz K, Buldu A. Detection of DDoS attacks with feed forward based deep neural network model. *Expert Syst Appl.* 2021;169:114520.
26. Yang XS, Hossein Gandomi A. Bat algorithm: a novel approach for global engineering optimization. *Eng Comput.* 2012;29(5):464-83.